本文档由 funstory.ai 的开源 PDF 翻译库 BabelDOC v0.6.2 (https://github.com/funstory-ai/BabelDOC) 翻译，本仓库正在积极的建设当中，欢迎 star 和关注。

3D高斯喷溅实时辐射场渲染

BERNHARD KERBL*, Inria, 蔚蓝海岸大学, 法国 GEORGIOS KOPANAS*, Inria, 蔚蓝海岸大学, 法国 THOMAS LEIMKÜHLER, 马克斯·普朗克信息研究所, 德国 GEORGE DRETTAKIS,Inria, 蔚蓝海岸大学, 法国

图1。我们的方法实现了实时渲染辐射场，其质量与之前质量最好的方法 [Barron等人 2022], 相当，同时仅需优化时间，可与最快的之前方法 [Fridovich-Keil和余等人2022年；米勒等人 2022]相媲美。这一性能的关键在于一种新颖的3D高斯场景表示，结合实时可微分的渲染器，为场景优化和新视角合成提供了显著的速度提升。请注意，为了与InstantNGP [米勒等人 2022],具有可比的训练时间，我们实现了与它们相似的质量；虽然这是他们达到的最高质量，但通过训练51分钟，我们实现了最先进的质量，甚至略优于Mip-NeRF360 [Barron等人 2022]。

辐射场方法最近彻底改变了使用多张照片或视频拍摄的场景的新视角合成。然而，要实现高视觉效果仍然需要训练和渲染成本高昂的神经网络，而最近更快的方 法不可避免地以牺牲质量为代价来换取速度。对于无边界和完整的场景（而不是孤立物体）以及1080p分辨率渲染，目前没有任何方法能够实现实时显示速率。我们引入了三个关键要素，使我们能够在保持有竞争力的训练时间的同时实现最先进的视觉效果，更重要的是，能够在1080p分辨率下实现高质量实时（≥ 30 帧）的新视角合成。首先，从相机校准过程中产生的稀疏点开始，我们使用三维高斯来表示场景，这些高斯保留了连续体积辐射场在场景优化中所需的理想特性，同时避免了在空旷空间中进行不必要的计算；其次，我们对三维高斯进行交错优化/密度控制，值得注意的是优化各向异性协方差以实现对场景的精确表示；第三，我们开发了一种快速感知可见性的渲染算法，该算法支持各向异性喷溅，并加速了训练，同时允许实时渲染。我们在几个已建立的数据库上展示了最先进的视觉效果和实时渲染。

CCS 概念：• **计算方法** → **渲染**；**基于点的模型**；**光栅化**；**机器学习方法**。

*两位作者对论文贡献相同。

作者地址：Bernhard Kerbl, bernhard.kerbl@inria.fr, Inria, 蔚蓝海岸大学, 法国；Georgios Kopanas, georgios.kopanas@inria.fr, Inria, 蔚蓝海岸大学, 法国；Thomas Leimkühler, thomas.leimkuehler@mpi-inf.mpg.de, 德国马普所信息研究所, 德国；George Drettakis, george.drettakis@inria.fr, Inria, 蔚蓝海岸大学, 法国。

出版权许可给ACM。ACM承认该贡献由政府雇员、承包商或附属机构撰写或共同撰写。因此，政府保留在政府目的下出版或复制本文的非排他性、免版税的权利，或允许他人这样做。© 2023 版权归所有者/作者所有。出版权许可给ACM。0730-0301/2023/8-ART1 $15.00 https://doi.org/10.1145/3592433

其他关键词和短语：新颖视图合成、辐射场、三维高斯、实时渲染

ACM参考文献格式：
Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, 和 George Dret-takis. 2023. 3D高斯喷溅用于实时辐射场渲染. *ACM Trans. Graph.* 42, 4, 文章1 (2023年8月), 14页. https: //doi.org/10.1145/3592433

1 引言

网格和点是最常见的3D场景表示方法，因为它们是显式的，并且非常适合快速基于GPU/CUDA的光栅化。相比之下，最近的神经辐射场（NeRF）方法基于连续场景表示，通常使用体积光线追踪优化多层感知器（MLP），以合成捕获场景的新视角。类似地，迄今为止最有效的辐射场解决方案都基于连续表示，通过插值存储在例如体素 [Fridovich-Keil 和余等人 2022] 或哈希 [米勒等人 2022] 网格或点 [徐等人 2022] 中。虽然这些方法的连续性有助于优化，但渲染所需的随机采样成本高昂，并可能导致噪声。我们介绍了一种结合两者优点的新方法：我们的3D高斯表示允许使用最先进的（SOTA）视觉质量和有竞争力的训练时间进行优化，而我们的基于瓦片的喷溅解决方案确保在多个先前发表的数据集 [Barron等人2022年; Hedman等人2018年; Knapitsch等人 2017] (见图1) 上以SOTA质量实时渲染1080p分辨率。

我们的目标是实现对由多张照片捕获的场景进行实时渲染，并创建优化时间与典型真实场景中最有效的前置方法一样快的表示。最近的方法实现了快速训练 [Fridovich-Keil

和余等人2022年；米勒等人 2022], , 但难以达到当前SOTA NeRF方法所获得的视觉效果，即Mip-NeRF360 [巴隆等人 2022], , 该方法需要长达48小时的训练时间。快速—但质量较低—的辐射场方法可以实现交互式渲染时间，具体取决于场景（每秒10-15帧），但在高分辨率下难以实现实时渲染。

我们的解决方案基于三个主要组件。我们首先引入 三维高斯 作为一种灵活且表达能力强的场景表示方法。我们使用与先前 NeRF 类似方法相同的输入，即经过运动结构重建（SfM） [Snavely 等人.2006]校准的相机，并使用 SfM 过程中免费生成的稀疏点云初始化三维高斯集。与大多数需要多视图立体（MVS）数据 [Aliev 等人 2020 年；Kopanas 等人 2021 年；Rückert 等人 2022], 的点云解决方案不同，我们仅使用 SfM 点作为输入即可实现高质量结果。请注意，对于 NeRF 合成数据集，我们的方法即使使用随机初始化也能实现高质量。我们证明三维高斯是一个极好的选择，因为它们是一种可微分的体积表示，但也可以通过将它们投影到 2D 并应用标准 α-混合，使用与 NeRF 相同的等效图像形成模型来非常高效地光栅化。我们方法中的第二个组件是三维高斯属性的优化——三维位置、不透明度 α、各向异性协方差和球谐（SH）系数——与自适应密度控制步骤交错进行，我们在优化过程中会添加和偶尔删除三维高斯。优化过程产生了一个合理紧凑、非结构化和精确的场景表示（所有测试场景为 100 万至 500 万个高斯）。我们方法中的第三个也是最后一个元素是我们的实时渲染解决方案，该方案使用快速 GPU 排序算法，并受基于瓦片的光栅化启发，遵循最近的工作 [Lassner 和 Zollhofer 2021]。然而，由于我们的三维高斯表示，我们可以执行各向异性喷溅，该喷溅尊重可见性排序——得益于排序和 α-混合——并通过跟踪所需数量的排序后喷溅的遍历来实现快速且精确的反向传播。

总而言之，我们提供了以下贡献：

- 将各向异性三维高斯作为高质量、非结构化辐射场表示的引入。
- 一种三维高斯属性优化方法，结合自适应密度控制，为捕获场景创建高质量表示。
- 一种用于GPU的快速、可微分的渲染方法，具有可见性感知、支持各向异性喷溅和快速反向传播，以实现高质量的新视角合成。

我们在先前发表的数据集上的结果表明，我们可以从多视角捕获中优化我们的三维高斯，并达到或超过先前最佳隐式辐射场方法的最高质量。我们还可以达到与最快方法相似的训练速度和质量，重要的是，我们提供了首个具有高质量的新视角合成的<样式 id='1'>实时渲染</样式>。

2 相关工作

我们首先简要概述传统重建，然后讨论基于点的渲染和辐射场工作，探讨它们的相似性；辐射场是一个广阔的领域，因此我们只关注与之直接相关的工作。若要全面了解该领域，请参阅 [Tewari等人2022年；Xie等人 2022]的出色近期综述。

2.1 传统场景重建与渲染

首批新颖视图合成方法基于光场，首先由Gortler等人于1996年密集采样 [；随后Levoy和Hanrahan 1996] 允许非结构化采集 [Buehler等人 2001]。运动结构重建（SfM） [Snavely等人 2006] 的出现开辟了一个全新领域，其中一组照片可用于合成新颖视图。SfM在相机标定过程中估计稀疏点云，最初用于简单可视化三维空间。随后的多视图立体（MVS） [Goesele等人 2007], 在多年间发展出令人印象深刻的完整三维重建算法，使多种视图合成算法 [Chaurasia等人2013；Eisemann等人 2008；Hedman等人2018；Kopanas等人 2021]的开发成为可能。所有这些方法<样式 id='11'>重新投影 </样式>和<样式 id='13'>混合 </样式>输入图像到新颖视图相机，并使用几何信息指导这种重新投影。这些方法在许多情况下产生了优异的结果，但通常无法完全恢复未重建区域，或从“过重建”中恢复，当MVS生成不存在几何时。最新的神经渲染算法 [Tewari等人 2022] 大幅减少了此类伪影，并避免了在GPU上存储所有输入图像的巨大成本，在大多数方面超越了这些方法。

2.2 神经渲染与辐射场

深度学习技术很早就被用于新视角合成 [Flynn 等人 2016；Zhou 等人 2016]；CNNs 被用于估计混合权重 [Hedman 等人 2018], 或用于纹理空间解决方案 [Riegler 和 Koltun 2020；Thies 等人 2019]。基于 MVS 的几何是这些方法中的主要缺点；此外，使用 CNNs 进行最终渲染经常导致时间闪烁。

体积表示用于新视角合成是由 Soft3D [Penner 和 Zhang 2017]发起的；随后提出了结合体积光线追踪的深度学习技术 [Henzleret al. 2019；Sitzmann 等人 2019]在连续可微密度场的基础上构建几何表示。使用体积光线追踪渲染由于需要大量采样查询体积而具有显著成本。神经辐射场（NeRFs） [Mildenhall 等人 2020] 引入了重要性采样和位置编码来提高质量，但使用了大型多层感知器从而影响速度。NeRF 的成功导致了大量后续方法涌现，这些方法通过引入正则化策略来解决质量和速度问题；目前新视角合成的图像质量最先进技术是 Mip-NeRF360 [Barron 等人 2022]。虽然渲染质量出色，但训练和渲染时间仍然极高；我们能够在某些情况下达到或超越这种质量，同时提供快速训练和实时渲染。

最新的方法主要关注更快的训练和/或渲染，这主要通过三种设计选择实现：使用空间数据结构来存储（神经）特征，这些特征随后在体积光线追踪过程中进行插值；不同的编码方式；以及 MLP 容量。

这类方法包括空间离散化的不同变体 [陈等人 2022b，a；弗里多维奇-凯尔和余等人 2022；加尔宾等人 2021；黑德曼等人 2021；赖瑟等人 2021；高木等人 2021；吴等人 2022；余等人 2021], 码本 [高木等人 2022], 以及哈希表 [米勒等人 2022]„ 允许使用更小的 MLP 或完全放弃神经网络 [弗里多维奇-凯尔和余等人 2022；孙等人 2022]。

这些方法中最值得注意的是InstantNGP [米勒等人 2022], 它使用哈希网格和占用网格来加速计算，并使用更小的MLP来表示密度和外观；以及Plenoxels [弗里多维奇-凯尔和余等人 2022] ，它们使用稀疏体素网格来插值连续密度场，并且能够完全放弃神经网络。两者都依赖于球谐函数：前者直接表示方向性效果，后者将其输入编码到颜色网络中。虽然这两种方法都能提供出色的结果，但这些方法在表示空旷空间时仍然可能存在困难，部分取决于场景/捕捉类型。此外，图像质量在很大程度上受限于用于加速的结构化网格的选择，而渲染速度则受到需要为给定光线步进查询许多样本的阻碍。我们使用的无结构、显式的GPU友好三维高斯能够实现更快的渲染速度和更好的质量无需神经网络组件。

2.3 基于点的渲染与辐射场

基于点的渲染方法高效地渲染不连接且无结构的几何样本（即点云） [Gross和Pfister 2011]。在其最简单形式中，点样本渲染 [Grossman和Dally1998] 对一组无结构点进行光栅化，其具有固定大小，因此它可能利用图形API[Sainz和Pajarola 2004] 原生支持的点类型或GPU上的并行软件光栅化 [Laine和Karras 2011；Schütz等人 2022]。虽然忠实于底层数据，但点样本渲染存在孔洞、导致走样，并且严格不连续。关于高质量基于点的渲染的开创性工作通过用比像素范围更大的点原语进行“喷溅”来解决这些问题，例如圆形或椭圆形盘、椭球体或surfels [Botsch等人 2005；Pfister等人 2000；Ren等人 2002；Zwicker等人 2001b]。

最近对 可微分的基于点的渲染技术 [Wiles 等人 2020；Yifan 等人 2019] 感兴趣。点被增加了神经特征并使用 CNN [Aliev 等人 2020；Rückert 等人 2022] 进行渲染，从而实现快速甚至实时视图合成；然而它们仍然依赖于 MVS 进行初始几何构建，因此继承了其伪影，尤其是在特征缺失/反光区域或细小结构等硬情况下，存在过重建或欠重建问题。

基于点的 α-混合与 NeRF 风格的体积渲染本质上使用相同的图像形成模型。具体来说，颜色 C 由沿光线进行的体积渲染给出：

$$C = \sum_{i=1}^{N} T_i(1-\exp(-\sigma_i\delta_i))\mathbf{c}_i \quad \text{with} \quad T_i = \exp\left(-\sum_{j=1}^{i-1}\sigma_j\delta_j\right), \tag{1}$$

其中沿光线以间隔 δ_i 采样密度 σ、透射率 T 和颜色 $\mathbf{c}$，这可以重写为

$$C = \sum_{i=1}^{N} T_i\alpha_i\mathbf{c}_i, \tag{2}$$

with

$$\alpha_i = (1-\exp(-\sigma_i\delta_i)) \text{ and } T_i = \prod_{j=1}^{i-1}(1-\alpha_i).$$

一种典型的神经点云方法（例如， [Kopanas 等人 2022,2021]）通过混合 N 个有序且与像素重叠的点来计算像素的颜色 C：

$$C = \sum_{i\in N} c_i\alpha_i\prod_{j=1}^{i-1}(1-\alpha_j), \tag{3}$$

其中 $\mathbf{c}_i$ 是每个点的颜色，而 α_i 是通过使用协方差 Σ [对 2D 高斯函数进行评估后，再乘以学习到的每点不透明度 Yifan 等人 2019] 得到的。

从公式2和公式3中，我们可以清晰地看到图像形成模型是相同的。然而，渲染算法非常不同。NeRFs是一种连续表示，隐式地表示空/占用空间；需要昂贵的随机采样来在公式2中找到样本，从而导致噪声和计算成本。相比之下，点是 一种无结构、离散的表示，足够灵活，允许创建、销毁和移动类似于NeRF的几何体。这是通过优化不透明度和位置来实现的，如先前工作 [Kopanas等人 2021], 所示，同时避免了全体积表示的缺点。

脉冲星 [Lassner 和 Zollhofer 2021] 实现了快速 球体 光栅化，这启发了我们的基于瓦片和排序的渲染器。然而，根据上述分析，我们希望在排序后的喷溅上保持（近似）传统 α-混合，以获得体积表示的优势：我们的光栅化尊重可见性顺序，而他们的方法是无序的。此外，我们在像素中反向传播所有喷溅的梯度，并光栅化各向异性喷溅。所有这些元素都促成了我们结果的高视觉质量（见第 7.3 节）。此外，上述先前方法也使用 CNN 进行渲染，这导致时间不稳定性。尽管如此，脉冲星 [Lassner 和 Zollhofer 2021] 以及 ADOP [Rückert 等人 2022] 的渲染速度成为了我们开发快速渲染解决方案的动机。

在关注镜面效果的同时，神经点猫眼（Neural Point Catacaustics） [科帕纳斯（Kopanas）等人 2022]的漫反射点渲染技术克服了时间不稳定性，通过使用MLP，但仍需要MVS几何体作为输入。该类别中最新的方法 [张等人 2022] 不需要MVS，并且也使用SH表示方向；然而，它只能处理单个物体的场景，并且需要掩码进行初始化。虽然对于低分辨率和小点数场景速度快，但尚不清楚它如何扩展到典型数据集 [巴隆等人（Barron et al.）2022；黑德曼等人（Hedman et al.）2018；克纳普奇等人（Knapitsch et al.） 2017] 的场景。我们使用三维高斯（3D Gaussians）进行更灵活的场景表示，避免了MVS几何体的需求，并由于我们针对投影高斯体的基于瓦片的渲染算法，实现了实时渲染。

一种最近的方法 [徐等人 2022] 使用点通过径向基函数方法表示辐射场。他们在优化过程中采用点剪枝和密度增强技术，但使用体积光线追踪（volumetric ray-marching），无法实现实时显示速率。

在人体动作捕捉领域，三维高斯已被用于表示捕捉到的人体 [Rhodin 等人 2015；Stoll 等人 2011]；最近它们被用于体积光线追踪进行视觉任务 [Wang 等人 2023]。神经体积基元在类似背景下被提出 [Lombardi 等人 2021]。虽然这些方法启发了我们选择三维高斯作为场景表示，但它们专注于重建和渲染单个孤立对象（人体或面部）的特定情况，导致场景深度复杂度较小。相比之下，我们对 各向异性 协方差、我们的交错优化/密度控制以及渲染的深度排序优化，使我们能够处理包括背景在内的完整复杂场景，无论是室内还是室外，且具有较大的深度复杂度。

3 概述

我们方法的输入是一组静态场景的图像，以及由 SfM [Schönberger 和 Frahm 2016] 校准的对应相机，后者会产生一个稀疏点云作为副作用。从这些点中，我们创建了一组三维高斯（第 4 节），由位置（均值）、协方差矩阵和不透明度 α 定义，这允许非常灵活的优化机制。这导致了对 3D 场景的合理紧凑表示，部分原因是可以使用各向异性体积贴花来紧凑地表示精细结构。辐射场的方向外观分量（颜色）通过球谐系数（SH）表示，遵循标准做法 [Fridovich-Keil 和 Yu 等人 2022 年；米勒等人 2022]。我们的算法通过一系列三维高斯参数（即位置、协方差、 α 和球谐系数）的优化步骤来创建辐射场表示（第 5 节），这些步骤与高斯密度自适应控制的操作交替进行。我们方法效率的关键是我们基于瓦片的光栅化器（第 6 节），它允许 α-混合各向异性贴花，并通过快速排序尊重可见性顺序。我们的快速光栅化器还包括通过跟踪累积 α 值进行快速反向传递，而没有高斯可以接收梯度的限制。我们方法的概述在图 2 中说明。

4 可微分的 3D 高斯 Splatting

我们的目标是为从没有法线的稀疏（SfM）点集开始，优化一个场景表示，以实现高质量的全新视角合成。为此，我们需要一个既能继承可微分体积表示的属性，同时又是非结构化和显式的原始元素，以便实现非常快速的渲染。我们选择了 3D 高斯，它们是可微分的，并且可以轻松投影到 2D splats，从而允许快速 α-混合进行渲染。

我们的表示与之前使用二维点的方法相似 [Kopanas等人 2021年；Yifan等人 2019]，并假设每个点都是一个带有法线的小平面圆。鉴于SfM点的极端稀疏性，估计法线非常困难。类似地，从这种估计中优化非常嘈杂的法线将是非常具有挑战性的。相反，我们将几何形状建模为一系列不需要法线的三维高斯。我们的高斯由一个完整的3D协方差矩阵 Σ 定义，该矩阵在世界空间 [Zwicker等人2001a] 中以其中心点（均值） μ为中心：

$$G(x) = e^{-\frac{1}{2}(x)^T\Sigma^{-1}(x)} \tag{4}$$

。这个高斯在我们的混合过程中被乘以 α。

然而，我们需要将我们的 3D 高斯投影到 2D 以进行渲染。Zwicker 等人 [2001a] demonstrate 如何进行这种投影到图像空间。给定一个视点变换 W，相机坐标系中的协方差矩阵 Σ，表示如下：

$$\Sigma' = JW\Sigma W^T J^T \tag{5}$$

其中 J 是投影变换的仿射近似的雅可比矩阵。Zwicker 等人 [2001a] 还表明，如果我们跳过 Σ，的第三行和列，我们会得到一个具有相同结构和属性的 2×2 方差矩阵，就像我们从带有法线的平面点开始一样，就像先前的工作 [Kopanas 等人 2021] 中那样。

一个显而易见的方法是直接优化协方差矩阵 Σ，以获得表示辐射场的三维高斯。然而，只有当协方差矩阵是正半定的时候，它才具有物理意义。对于我们所有参数的优化，我们使用梯度下降，这种下降很难被约束以产生这样的有效矩阵，并且更新步骤和梯度很容易创建无效的协方差矩阵。

因此，我们选择了一种更直观、但同样表达力强的优化表示。三维高斯 Σ 的协方差矩阵类似于描述椭球体的配置。给定一个缩放矩阵 S 和旋转矩阵 R，我们可以找到相应的 Σ:

$$\Sigma = RSS^T R^T \tag{6}$$

为了允许对这两个因素进行独立优化，我们将它们分别存储：一个三维向量 s 用于缩放，一个四元数 q 来表示旋转。这些可以简单地转换为它们各自的矩阵并组合，确保归一化 q 以获得一个有效的单位四元数。

为了避免在训练过程中由于自动微分导致的大量开销，我们显式地推导出所有参数的梯度。精确的导数计算细节在附录 A 中。

这种各向异性协方差表示——适合优化——使我们能够优化三维高斯，以适应不同形状在捕获场景中的几何形状，从而得到一个相当紧凑的表示。图3说明了这种情况。

5 带自适应密度控制的3D高斯优化

我们方法的核心是优化步骤，该步骤创建了一组密集的三维高斯，精确地表示场景以用于自由视角合成。除了位置 p、 α 和协方差 Σ 之外，我们还优化每个高斯的球谐系数来表示颜色 c，以正确捕捉场景的视角依赖外观。这些参数的优化与控制高斯密度以更好地表示场景的步骤交替进行。

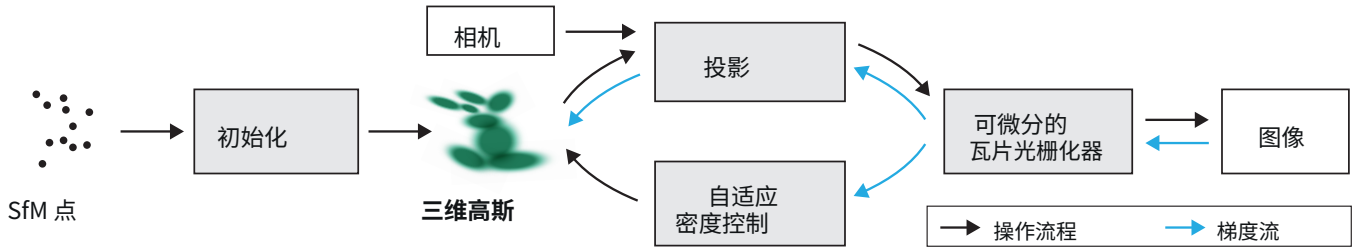

图2。优化从稀疏SfM点云开始，创建一组3D高斯。然后我们优化并自适应控制这组高斯的密度。在优化过程中，我们使用我们的快速基于瓦片的渲染器，与最先进快速辐射场方法相比，实现了有竞争力的训练时间。训练完成后，我们的渲染器允许在各种场景中进行实时导航。

图3。我们通过将它们缩小60%（最右侧）来可视化优化后的3D高斯。这清楚地显示了3D高斯的各向异性形状，这些形状在优化后紧凑地表示复杂几何形状。左侧是实际渲染的图像。

5.1 优化

优化基于渲染结果的图像与捕获数据集中训练视图的连续迭代比较。由于3D到2D投影的歧义性，几何结构不可避免地可能被错误放置。因此，我们的优化需要能够创建几何结构，并且也能够销毁或移动几何结构，如果它已经被错误放置。三维高斯的协方差参数的质量对于表示的紧凑性至关重要，因为大面积的均匀区域可以用少量大的各向异性高斯来捕获。

我们使用随机梯度下降技术进行优化，充分利用标准的GPU加速框架，并在某些操作中添加自定义CUDA核，遵循最近的最佳实践 [Fridovich-Keil 和余等人 2022；Sun 等人 2022]。特别是，我们的快速光栅化（见第6节）对我们的优化效率至关重要，因为它是最主要的计算瓶颈。

我们使用Sigmoid激活函数对 α 进行约束，使其在 $[0-1)$ 范围内获得平滑的梯度，并使用指数激活函数对协方差尺度进行类似的处理。

我们估计初始协方差矩阵为一个各向同性高斯分布，其轴等于到最近的三个点的距离的平均值。我们使用一种标准的指数衰减调度技术，类似于 Plenoxels [Fridovich-Keil 和余等人 2022], ，但仅用于位置。损失函数是 L_1 ，结合了 D-SSIM 项：

$$\mathcal{L} = (1-\lambda)\mathcal{L}_1 + \lambda\mathcal{L}_{\text{D-SSIM}} \tag{7}$$

我们在所有测试中都使用 $\lambda = 0.2$ 。我们在第 7.1 节中提供了学习调度和其他元素的具体细节。

5.2 高斯自适应控制

我们从SfM的初始稀疏点集开始，然后应用我们的方法自适应地控制单位体积内的高斯数量及其密度[1]，使我们能够从初始稀疏的高斯集过渡到更密集的集合并更好地表示场景，并具有正确的参数。在优化预热（见第7.1节）后，我们每100次迭代进行密集化，并移除任何本质上透明的任何高斯，即 α 小于阈值 ϵ_α。

我们的自适应高斯控制需要填充空白区域。它关注缺少几何特征的区域（“过重建”），但也包括高斯覆盖场景大面积的区域（通常对应“过重建”）。我们观察到这两种情况都具有 较大的视图空间位置梯度。直观上，这可能是因为它们对应尚未良好重建的区域，优化尝试将高斯移动以纠正这一点。

由于这两种情况都是密化（densification）的良好候选者，我们使用视图空间位置梯度平均幅度高于阈值τpos的高斯进行密化，我们在测试中将此阈值设置为0.0002。

我们接下来展示此过程的细节，如图4所示。

对于位于过重建区域的小高斯，我们需要覆盖必须创建的新几何形状。为此，最好通过简单地创建相同大小的副本并将其移动到位置梯度的方向来克隆高斯。

另一方面，高方差区域中的大型高斯分布需要被分割成更小的分布。我们用两个新的分布来替换这些分布，并将它们的尺度通过一个由我们通过实验确定的因子 $\phi = 1.6$ 进行缩放。我们还通过将原始的3D高斯分布作为概率密度函数来初始化它们的位置。

在第一种情况下，我们检测并处理需要增加系统总体积和高斯分布数量的需求，而在第二种情况下，我们保持总体积不变但增加高斯分布数量。与其他体积表示方法类似，我们的优化可能会被靠近输入相机的浮动点卡住；在我们的情况下，这可能导致高斯密度的不合理增加。一种有效的方法来调节高斯分布数量的增加是每隔 $N = 3000$将 α 值设置为接近零。

[1]高斯分布的密度当然不应与NeRF文献中的密度 σ 相混淆。

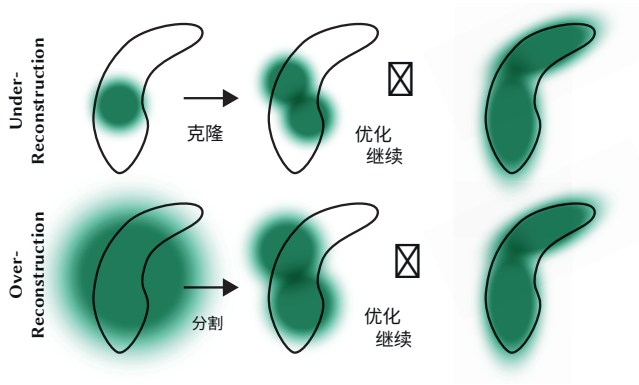

图4。我们的自适应高斯密度增强方案。顶行（欠重建）：当小尺度几何（黑色轮廓）覆盖不足时，我们克隆相应的高斯分布。底行（过重建）：如果小尺度几何由一个大型splat表示，我们将它分割成两个。

迭代次数。然后，优化会为需要的高斯分布增加 α，同时允许我们的剔除方法移除上述描述的 α 小于 ϵ_α 的高斯分布。高斯分布可能会缩小或增长，并与其他高斯分布显著重叠，但我们定期移除在世界空间中非常大以及在视图空间中占用很大空间的高斯分布。这种策略对高斯分布的总数提供了良好的控制。我们模型中的高斯分布始终是欧几里得空间中的基本元素；与其他方法（如Barron等人2022年；Fridovich-Keil和余等人）不同，我们不需要对远距离或较大的高斯分布进行空间压缩、变形或投影策略。

6 快速可微分的 高斯分布 光栅化器

我们的目标是实现快速整体渲染和快速排序，以允许近似 α-混合——包括各向异性splat——并避免先前工作中 [Lassner 和 Zollhofer 2021] 存在的梯度接收上限。

为实现这些目标，我们设计了一种基于瓦片的高斯splat光栅化器，灵感来自近期软件光栅化方法 [Lass- ner 和 Zollhofer 2021]，以一次对整个图像的图元进行预排序，避免了按像素排序的开销，从而解决了先前 α-混合方案 [Kopanas 等人 2022, 2021] 的限制。我们的快速光栅化器允许对任意数量的混合高斯分布进行高效反向传播，且附加内存消耗低，每个像素仅需常数开销。我们的光栅化流程是完全可微分的，结合投影到2D（第4节），可以像先前2D splatting方法 [Kopanas 等人 2021] 一样光栅化各向异性splat。

我们的方法首先将屏幕分割成 16×16 个瓦片，然后继续对视锥体和每个瓦片进行剔除三维高斯。具体来说，我们只保留置信区间与视锥体相交的99%高斯分布。此外，我们使用一个防护带来简单拒绝处于极端位置的高斯（即均值为近平面且远离视锥体的高斯），因为计算它们的投影2D协方差会不稳定。然后，我们根据它们重叠的瓦片数量实例化每个高斯，并为每个实例分配一个结合视空间深度和瓦片ID的键。然后，我们使用单个快速GPU基数排序 16×16 Merrill和Grimshaw根据这些键对高斯进行排序。请注意，没有额外的每像素或点排序，并且混合是基于初始排序执行的。因此，我们的混合在某些配置中可以是近似的。然而，随着splat接近单个像素的大小，这些近似变得可以忽略不计。我们发现这种选择大大提高了训练和渲染性能，而不会在收敛场景中产生可见伪影。

对高斯分布排序后，我们为每个瓦片生成一个列表，通过识别第一个和最后一个按深度排序且投射到给定瓦片的条目。在光栅化过程中，我们为每个瓦片启动一个线程块。每个线程块首先协作地将高斯分布的数据包加载到共享内存中，然后，对于给定像素，通过从前到后遍历列表来累积颜色和 α 值，从而最大化数据加载/共享和处理方面的并行性。当我们在像素中达到目标饱和度 α 时，相应的线程停止。在固定的时间间隔内，会查询瓦片中的线程，当所有像素都饱和（即 α 变为1）时，整个瓦片的处理结束。排序的详细信息和整体光栅化方法的概述在附录C中给出。

在光栅化过程中，α 的饱和度是唯一的停止条件。与之前的工作不同，我们不限制接收梯度更新的混合基元的数量。我们强制执行此属性，以允许我们的方法处理具有任意、变化深度复杂度的场景，并准确学习它们，而无需依赖特定场景的超参数调整。在反向传递过程中，因此我们必须恢复正向传递中每个像素的混合点序列。一种解决方案是存储全局内存中每个像素的任意长度的混合点列表 [Kopanas et al. 2021]。为了避免隐含的动态内存管理开销，我们选择再次遍历每个瓦片列表；我们可以重用正向传递中的高斯分布排序数组和瓦片范围。为了便于梯度计算，我们现在从后向前遍历它们。

遍历从影响瓦片中任何像素的最后一个点开始，并且将点加载到共享内存的过程再次协同进行。此外，每个像素只有在它们的深度小于或等于在正向传递中为其颜色做出贡献的最后一个点的深度时，才会开始（昂贵的）重叠测试和点处理。计算第4节中描述的梯度需要原始混合过程中每一步累积的不透明度值。而不是在反向传递中遍历一个显式且逐渐缩小的不透明度列表，我们可以通过仅在前向传递结束时存储总累积不透明度来恢复这些中间不透明度。具体来说，每个点存储前向过程中的最终累积不透明度 α；我们在从后到前的遍历中将此值除以每个点的 α，以获得梯度计算所需的系数。

7 实现与结果评估

我们接下来讨论一些实现细节，展示结果，并比较我们的算法与先前工作的评估以及消融研究。

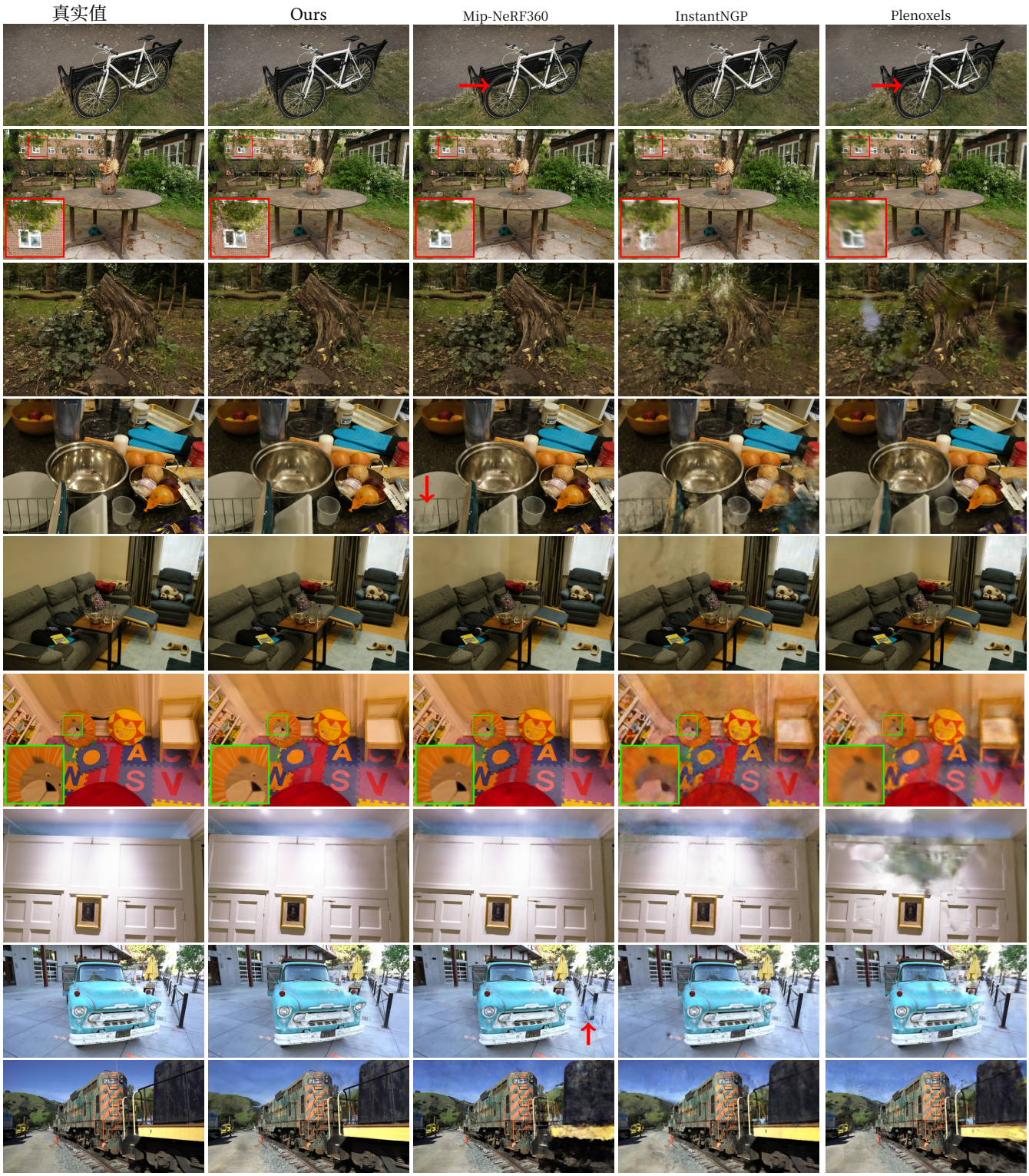

图5。我们展示了我们与先前方法的对比，以及从保留的测试视图中获取的对应真实值图像。场景从上到下依次为：来自Mip-NeRF360数据集的自行车、花园、树桩、计数器和房间；来自Deep Blending数据集的游乐园、DrJohnson [Hedman等人2018]，以及来自坦克与寺庙的卡车和火车。箭头/插入图中突出显示了非明显的质量差异。

表1。与先前工作的定量评估结果对比，该结果基于三个数据集计算得出。带剑号 † 的结果直接采用自原始论文，其余均为我们实验获得。

数据集	Mip-NeRF360						坦克与寺庙						深度融合					
方法指标	*SSIM*↑	*PSNR*↑	*LPIPS*↓	训练	FPS	Mem	*SSIM*↑	*PSNR*↑	*LPIPS*↓	训练	FPS	Mem	*SSIM*↑	*PSNR*↑	*LPIPS*↓	训练	FPS	Mem
Plenoxels	0.626	23.08	0.463	25分49秒	6.79	2.1GB	0.719	21.08	0.379	25分5秒	13.0	2.3GB	0.795	23.06	0.510	27分49秒	11.2	2.7GB
INGP-Base	0.671	25.30	0.371	5m37s	11.7	13MB	0.723	21.72	0.330	5m26s	17.1	13MB	0.797	23.62	0.423	6m31s	3.26	13MB
INGP-Big	0.699	25.59	0.331	7m30s	9.43	48MB	0.745	21.92	0.305	6m59s	14.4	48MB	0.817	24.96	0.390	8m	2.79	48MB
M-NeRF360	0.792†	27.69†	0.237†	48h	0.06	8.6MB	0.759	22.22	0.257	48h	0.14	8.6MB	0.901	29.40	0.245	48h	0.09	8.6MB
Ours-7K	0.770	25.60	0.279	6分25秒	160	523MB	0.767	21.20	0.280	6分55秒	197	270MB	0.875	27.78	0.317	4分35秒	172	386MB
Ours-30K	0.815	27.21	0.214	41m33s	134	734 MB	0.841	23.14	0.183	26m54s	154	411MB	0.903	29.41	0.243	36m2s	137	676 MB

图6。对于某些场景（上图），我们可以看到即使迭代到7K次（该场景的~5min），我们的方法也很好地捕捉到了列车。在30K次迭代（~35min）时，背景伪影已显著减少。对于其他场景（下图），差异几乎看不见；7K次迭代（~8min）已经非常高质量。

表2。合成神经辐射场的PSNR分数，我们从100K个随机初始化的点开始。从各自论文中提取的竞争指标。

	Mic	椅子	Ship	材料	Lego	鼓	Ficus	热狗	Avg.
Plenoxels	33.26	33.98	29.62	29.14	34.10	25.35	31.83	36.81	31.76
INGP-Base	36.22	35.00	31.10	29.78	36.39	26.02	33.51	37.40	33.18
Mip-NeRF	36.51	35.14	30.41	30.71	35.70	25.48	33.29	37.48	33.09
Point-NeRF	35.95	35.40	30.97	29.61	35.04	26.06	36.13	37.30	33.30
Ours-30K	35.36	35.83	30.80	30.00	35.78	26.15	34.87	37.72	33.32

7.1 实现方案

我们使用Python语言和PyTorch框架实现了我们的方法，并编写了用于光栅化的自定义CUDA核，这些核是先前方法的扩展版本 [Kopanas等人 2021], 以及使用NVIDIA CUB排序例程进行快速基数排序 [Mer- rill和Grimshaw 2010]。我们还使用开源的SIBR构建了一个交互式查看器 [Bonopera等人 2020], 用于交互式查看。我们使用此实现方案来测量我们实现的帧率。源代码和所有我们的数据都可以在以下地址获取：

https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/

优化细节。为了稳定性，我们在低分辨率下“预热”计算。具体来说，我们使用4倍于图像分辨率更小的优化，并在250次和500次迭代后进行两次上采样。

球谐系数优化对缺乏角度信息很敏感。对于典型的“NeRF-like”捕捉，其中中心物体被围绕它的整个半球拍摄的照片所观察，优化效果很好。然而，如果捕捉存在角度区域缺失（例如，在捕捉场景角落时，或执行“内向外” [Hedman et al. 2016] 捕捉），优化可能会产生SH零阶分量的完全错误值（即基础或漫反射颜色）可以由优化产生。为了克服这个问题，我们首先仅优化零阶分量，然后引入SH的一个波段，在每1000次迭代后，直到所有4个SH波段都被表示出来。

7.2 结果与评估

结果。我们在先前发表的数据集和合成Blender数据集 [Mildenhall等人 2020]中总共测试了13个真实场景。特别地，我们在Mip-Nerf360 [Barron等人 2022], 中呈现的所有场景集上测试了我们的方法，这是当前NeRF渲染质量的最先进水平，以及来自Tanks&Temples数据集 [2017] 的两个场景和Hedman等人 [Hedman等人 2018]提供的两个场景。我们选择的所有场景具有非常不同的捕捉风格，涵盖了有界室内场景和大型无界室外环境。我们在评估中所有实验都使用相同的超参数配置。所有结果均报告在A6000 GPU上运行，Mip-NeRF360方法除外（见下文）。

在补充材料中，我们展示了一段渲染视频路径，用于选择包含远离输入照片视角的场景。

真实世界场景。就质量而言，当前最先进的技术是Mip-NeRF 360 [Barron等人 2021]。我们将此方法作为质量基准进行比较。我们还与两种最新的快速NeRF方法进行了比较：InstantNGP [米勒等人 2022] 和Plenoxels [Fridovich-Keil与余等人 2022]。

我们使用训练/测试分割方法处理数据集，采用Mip-NeRF360建议的方法，每隔8张照片取一张用于测试，以进行一致且具有意义的比较，从而生成误差指标，使用文献中最常用的标准PSNR、L-PIPS和SSIM指标；请参见表1。表中的所有数值均来自我们对前人所有方法的作者代码的运行结果，除了Mip-NeRF360在其数据集上的结果，我们直接复制了原始发表中的数值，以避免对当前SOTA的混淆。对于我们图中的图像，我们使用了Mip-NeRF360的独立运行版本：这些运行结果的数值在附录D中。我们还展示了平均训练时间、渲染速度以及存储优化参数所需的内存。我们报告了InstantNGP（Base）的基本配置（运行35K次迭代）以及作者建议的稍大网络（Big），以及我们提出的两个配置：7K和30K次迭代。我们展示了

表 3. 消融实验的 PSNR 分数。在此实验中，我们手动将每个场景的输入图像的高分辨率版本下采样到我们其他实验的既定渲染分辨率。这样做减少了随机伪影（例如，由于预先下采样的 Mip-NeRF360 输入中的 JPEG 压缩）。

	卡车-5K	花园-5K	自行车-5K	卡车-30K	花园-30K	自行车-30K	平均-5K	平均-30K
有限带宽	14.66	22.07	20.77	13.84	22.88	20.87	19.16	19.19
随机初始化	16.75	20.90	19.86	18.02	22.19	21.05	19.17	20.42
不分割	18.31	23.98	22.21	20.59	26.11	25.02	21.50	23.90
不SH	22.36	25.22	22.88	24.39	26.59	25.08	23.48	25.35
不克隆	22.29	25.61	22.15	24.82	27.47	25.46	23.35	25.91
各向同性	22.40	25.49	22.81	23.89	27.00	24.81	23.56	25.23
Full	22.71	25.82	23.18	24.81	27.70	25.65	23.90	26.05

我们两种配置在图6中的视觉质量差异。在许多情况下，7K次迭代时的质量已经相当好。

训练时间因数据集而异，我们分别报告。请注意，图像分辨率也因数据集而异。在项目网站上，我们提供了所有用于计算所有方法（包括我们和先前工作）在所有场景中统计数据测试视图的渲染结果。请注意，所有渲染均保持了原始输入分辨率。

表格显示，我们的完全收敛模型在质量上与SOTA Mip-NeRF360方法相当，有时略好；请注意，在相同硬件上，他们的平均训练时间为48小时[2]，而我们的为35-45分钟，他们的渲染时间为每帧10秒。经过5-10分钟的训练，我们在质量上与InstantNGP和Plenoxels相当，但额外的训练时间使我们能够达到SOTA质量，而其他快速方法则不能。对于Tanks & Temples，我们在相似的训练时间（我们情况下的~7分钟）下达到了与基本InstantNGP相似的渲染质量。

我们还展示了针对我们和图5中选定的先前渲染方法进行的留出测试视图的比较可视化结果；我们方法的结果是基于30K次训练迭代。我们看到，在某些情况下，即使Mip-NeRF360也存在我们方法避免的残留伪影（例如，植被中的模糊——在自行车、树桩中——或房间墙壁上的模糊）。在补充视频和网页上，我们提供了从远处比较路径的结果。我们方法倾向于从远处保留覆盖良好区域的视觉细节，而先前方法并非总是如此。

合成有界场景。 除了真实场景外，我们还评估了我们的方法在合成 *Blender* 数据集 [Mildenhall 等人 2020] 上的表现。这些场景提供了详尽的视角集合，尺寸有限，并提供精确的相机参数。在这样的场景中，即使使用随机初始化，我们也能实现最先进的结果：我们从包围场景边界的体积内均匀随机初始化 100K 个高斯分布开始训练。我们的方法可以快速自动地将其修剪到约 6–10K 个有意义的分布。经过 30K 次迭代后，训练模型的最终大小达到每个场景约 200–500K 个分布。我们使用白色背景以确保兼容性，在表 2 中报告并比较了与先前方法的 PSNR 分数。示例可以在图 10（左侧第二个图像）和补充材料中可见。训练的合成场景渲染帧率可达 180–300 FPS。

紧凑性。 与先前的显式场景表示相比，我们优化中使用的各向异性高斯能够用更少的参数建模复杂形状。我们通过将我们的方法与Zhang等人 [获得的紧凑的基于点的模型进行评估来展示这一点。我们从他们通过使用前景掩码进行空间雕刻获得的初始点云开始，直到我们打破他们报告的PSNR分数。这通常在2-4分钟内发生。我们使用大约他们点数四分之一，实现了平均模型大小为3.8 MB，而他们的模型为9 MB。我们注意到，对于这个实验，我们只使用了他们球形谐波的两个度，类似于他们。

7.3 消融实验

我们隔离了我们所做的不同贡献和算法选择，并构建了一组实验来衡量它们的效果。具体来说，我们测试了以下算法的方面：从SfM初始化、我们的稠密化策略、各向异性协方差、我们允许无限数量的splat具有梯度和使用球谐函数。每个选择的定量效果总结在表3中。

从*SfM*初始化。 我们还评估了从SfM点云初始化三维高斯的的重要性。对于这个消融实验，我们均匀采样一个边长等于输入相机包围盒范围三倍的立方体。我们观察到我们的方法表现相对较好，即使没有SfM点也不会完全失败。相反，它主要在背景中退化，见图7。此外，在训练视图覆盖不佳的区域，随机初始化方法似乎有更多浮动点，这些点无法通过优化去除。另一方面，合成的NeRF数据集没有这种行为，因为它没有背景，并且由输入相机很好地约束（见上文讨论）。

密集化。 接下来，我们评估了我们的两种密集化方法，更具体地说，是第5节中描述的克隆和分割策略。我们分别禁用每种方法，并使用其余方法不变进行优化。结果表明，分割大高斯分布对于允许背景的良好重建非常重要，如图8所示，而克隆小高斯分布而不是分割它们则允许更好的收敛速度，尤其是在场景中出现细结构时。

[2]我们在4-GPU A100节点上训练了Mip-NeRF360，持续12小时，相当于单GPU上训练48小时。请注意，A100比A6000 GPU更快。

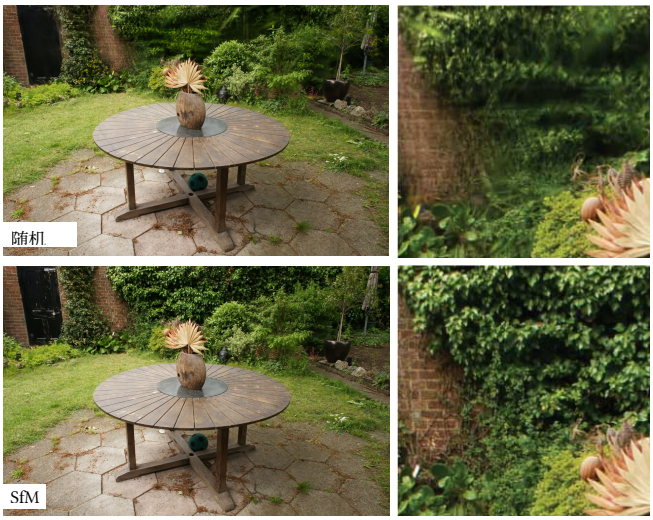

图7。使用SfM点进行初始化有帮助。上方：使用随机点云进行初始化。下方：使用SfM点进行初始化。

图8. “克隆”和“分割”（第5节）两种情况下，密化策略的消融实验。

无限深度，具有梯度的*splat*的复杂性。 我们评估了是否跳过在 N 最前面的点之后的梯度计算

图9. 如果限制接收梯度的点数，对视觉效果的影响非常显著。左：限制为10个高斯接收梯度。右：我们的完整方法。

这将让我们在牺牲质量的情况下获得速度，正如Pulsar [Lassner和Zollhofer 2021]所建议的那样。在本测试中，我们选择N=10，其值是Pulsar中默认值的两倍，但由于梯度计算中的严重近似，导致优化不稳定。对于卡车场景，PSNR（见表3，有限带宽）质量下降了11dB，视觉效果如图9（花园）所示。

各向异性协方差。 我们方法中的一个重要算法选择是优化三维高斯的完整协方差矩阵。为了展示这一选择的效果，我们进行了消融实验，通过优化一个控制三维高斯在三个轴上半径的单个标量值来移除各向异性。该优化的结果在图10中直观呈现。我们观察到，各向异性显著提高了三维高斯与表面对齐的能力，进而允许在保持相同点数的情况下实现更高的渲染质量。

球谐函数。 最后，使用球谐函数提高了我们的整体PSNR分数，因为它们补偿了视点相关的效果（表3）。

7.4 限制

我们的方法并非没有限制。在场景观察不佳的区域，我们会出现伪影；在这些区域，其他方法也难以处理（例如，图11中的Mip-NeRF360）。尽管各向异性高斯如上所述有许多优点，但我们的方法可能会产生细长的伪影或“斑驳”的高斯（见图12）；同样，之前的方法在这些情况下也难以处理。

我们还偶尔会出现弹出伪影，当我们的优化产生大型高斯时会发生这种情况；这通常发生在视点相关外观的区域。这些弹出伪影的一个原因是光栅化器中的防护带对高斯的简单拒绝。更原则性的剔除方法可以缓解这些伪影。另一个因素是我们简单的可见性算法，这可能导致高斯突然切换深度/混合顺序。这可以通过抗锯齿来解决，我们将其留作未来工作。此外，我们目前不对优化应用任何正则化；这样做将有助于未观察区域和弹出伪影。

尽管我们在完整评估中使用了相同的超参数，但早期实验表明，在非常大的场景（例如城市数据集）中收敛可能需要降低位置学习率。

图 10. 我们使用高斯训练场景对视觉效果有显著影响。不各向异性体积贴花的运用能够建模精细结构，并具有为说明目的，我们将Ficus限制在两种配置中最多使用5k个高斯分布。

尽管与之前的基于点的方案相比我们非常紧凑，但我们的内存消耗显著高于基于NeRF的解决方案。在大型场景的训练过程中，我们未优化的原型中峰值GPU内存消耗可能超过20 GB。然而，通过精心设计优化逻辑的低级实现（类似于InstantNGP），这一数字可以显著降低。渲染训练好的场景需要足够的GPU内存来存储完整模型（对于大规模场景，需要几百兆字节）以及额外的30–500 MB用于光栅化器，具体取决于场景大小和图像分辨率。我们注意到，我们的方法在进一步降低内存消耗方面有许多机会。点云的压缩技术是一个研究得很好的领域 [De Queiroz和Chou 2016]；有趣的是，我们可以看看如何将这些方法应用于我们的表示。

图11。失败伪影的比较：Mip-NeRF360有“浮动点”和颗粒状外观（左侧，前景），而我们的方法产生粗糙、各向异性的高斯分布，导致低细节视觉效果（右侧，背景）。DrJohnson场景。

图12。在与其他训练期间看到的场景重叠很少的视图中，我们的方法可能会产生伪影（右侧）。同样，Mip-NeRF360在这些情况下也有伪影（左侧）。DrJohnson场景。

8 讨论与结论

我们提出了第一个真正允许在多种场景和捕捉风格中进行实时、高质量辐射场渲染的方法，同时训练时间与最快的先前方法具有竞争力。

我们选择使用3D高斯原语，在优化过程中保留了体积渲染的特性，同时直接支持基于快速splat的光栅化。我们的工作证明——与普遍接受的观点相反——要实现快速且高质量的辐射场训练，连续表示并非严格必要。不是必需的。

我们训练时间的绝大多数（~80%）都用于Python代码，因为我们使用PyTorch构建解决方案，以便其他人能轻松使用我们的方法。只有光栅化例程以优化的CUDA核形式实现。我们预计，将剩余的优化完全移植到CUDA——例如InstantNGP [米勒等人 2022],所做的那样——将显著提升对性能要求关键的应用场景的速度。

我们还展示了基于实时渲染原理构建的重要性，利用GPU的强大功能和软件光栅化流水线的速度。这些设计选择是训练和实时渲染性能的关键，在性能上优于之前的体积光线追踪。

如果我们的 Gaussianscan 能否用于对捕获的场景进行网格重建，这将很有趣。除了考虑到网格的广泛应用所带来的实际意义外，这还将使我们能够更好地理解我们的方法在体积表示和表面表示之间的连续体中确切的位置。

总之，我们提出了首个实时渲染辐射场的解决方案，其渲染质量与最昂贵的先前方法相当，训练时间与最快的现有解决方案具有竞争力。

致谢

这项研究得到了ERC高级资助项目FUNGRAPH（编号788065）的资助http://fungraph.inria.fr。作者们感谢Adobe的慷慨捐赠，蔚蓝海岸大学的OPAL基础设施，以及来自GENCI–IDRIS（资助编号2022-AD011013409）的高性能计算资源。作者们感谢匿名审稿人的宝贵反馈，P. Hedman和A. Tewari对早期草稿的校对，以及T. Müller、A. Yu和S. Fridovich-Keil在比较工作中的帮助。

参考文献

Kara-Ali Aliev、Artem Sevastopolsky、Maria Kolos、Dmitry Ulyanov和Victor Lem-itsky。2020年。神经点云图形。收录于《计算机视觉——ECCV 2020：第16届欧洲会议，英国格拉斯哥，2020年8月23日至28日，会议论文集，第XXII部分》。第696–712页。Jonathan T Barron、Ben Mildenhall、Matthew Tancik、Peter Hedman、Ricardo Martin- Brualla和Pratul P Srinivasan。2021年。Mip-nerf：用于抗锯齿神经辐射场的多尺度表示。收录于《IEEE/CVF国际计算机视觉会议论文集》。第5855–5864页。Jonathan T. Barron、Ben Mildenhall、Dor Verbin、Pratul P. Srinivasan和Peter Hedman。2022年。Mip-NeRF 360：无限制的抗锯齿神经辐射场。*CVPR*（2022年）。Sebastien Bonopera、Jerome Esnault、Siddhant Prakash、Simon Rodriguez、Theo Thonat、Mehdi Benadel、Gaurav Chaurasia、Julien Philip和George Drettakis。2020年。sibr：基于图像渲染的系统。https://gitlab.inria.fr/sibr/sibr_core Mario Botsch、Alexander Hornung、Matthias Zwicker和Leif Kobbelt。2005年。当今GPU上的高质量表面喷溅。收录于《第二届Eurographics / IEEE VGTC点云图形会议论文集》（美国纽约）*(SPBG'05)*。欧洲图形学协会，德国戈斯拉尔，第17–24页。Chris Buehler、Michael Bosse、Leonard McMillan、Steven Gortler和Michael Cohen。2001年。非结构化发光图渲染。收录于《Proc. SIGGRAPH》。Gaurav Chaurasia、Sylvain Duchene、Olga Sorkine-Hornung和George Drettakis。2013年。深度合成和局部扭曲，用于逼真的基于图像导航。*ACM Transactions on Graphics (TOG)* 32, 3 (2013)，第1–12页。Anpei Chen、Zexiang Xu、Andreas Geiger、Jingyi Yu和Hao Su。2022b。TensoRF：张量辐射场。收录于《欧洲计算机视觉会议（ECCV）》。Zhiqin Chen、Thomas Funkhouser、Peter Hedman和Andrea Tagliasacchi。2022a。MobileNeRF：利用多边形光栅化管道在移动架构上高效渲染神经场。 *arXiv preprint arXiv:2208.00277*（2022年）。Ricardo L De Queiroz和Philip A Chou。2016年。使用区域自适应分层变换压缩3D点云。*IEEE Transactions on Image Processing* 25, 8 2016 3947 3956 ( )。Martin Eisemann、Bert De Decker、Marcus Magnor、Philippe Bekaert、Edilson De Aguiar、Naveed Ahmed、Christian Theobalt和Anita Sellent。2008年。浮点纹理。收录于《计算机图形学论坛》，第27卷。Wiley Online Library，第409–418页。John Flynn、Ivan Neulander、James Philbin和Noah Snavely。2016年。Deepstereo：从世界图像中学习预测新视角。收录于《CVPR》。Fridovich-Keil和Yu、Matthew Tancik、Qinhong Chen、Benjamin Recht和Angjoo Kanazawa。2022年。Plenoxels：无需神经网络的辐射场。*CVPR*。Stephan J. Garbin、Marek Kowalski、Matthew Johnson、Jamie Shotton和Julien Valentin。2021年。FastNeRF：200FPS的高保真神经渲染。收录于《IEEE/CVF国际计算机视觉会议论文集（ICCV）》第14346–14355页。Michael Goesele、Noah Snavely、Brian Curless、Hugues Hoppe和Steven M Seitz。2007年。用于社区照片集的多视图立体重建。收录于《ICCV》。Steven J Gortler、Radek Grzeszczuk、Richard Szeliski和Michael F Cohen。1996年。发光图。收录于《第23届计算机图形学和交互技术会议论文集》。第43–54页。Markus Gross和Hanspeter (Eds) Pfister。2011年。点云图形。Elsevier。Jeff P. Grossman和William J. Dally。1998年。点采样渲染。收录于《渲染技术》。Peter Hedman、Julien Philip、True Price、Jan-Michael Frahm、George Drettakis和Gabriel Brostow。2018年。用于自由视点基于图像渲染的深度融合。*ACM Trans. on Graphics (TOG)* 37, 6 (2018年)。Peter Hedman、Tobias Ritschel、George Drettakis和Gabriel Brostow。2016年。可扩展的向外基于图像渲染。*ACM Transactions on Graphics (SIGGRAPH Asia Conference Proceedings)* 35, 6 (2016年12月)。http://www-sop.inria.fr/reves/ Basilic/2016/HRDB16 Peter Hedman、Pratul P. Srinivasan、Ben Mildenhall、Jonathan T. Barron和Paul Debevec。2021年。烘焙神经辐射场以实现实时视图合成。*ICCV*（2021年）。Philipp Henzler、Niloy J Mitra和Tobias Ritschel。2019年。逃离柏拉图的洞穴：基于对抗渲染的3D形状。收录于《IEEE/CVF国际计算机视觉会议论文集》。第9984–9993页。Arno Knapitsch、Jaesik Park、Qian-Yi Zhou和Vladlen Koltun。2017年。坦克和寺庙：大规模场景重建基准测试。*ACM Transactions on Graphics (ToG)* 36, 4 (2017年)，第1–13页。Georgios Kopanas、Thomas Leimkühler、Gilles Rainer、Clément Jambon和George Drettakis。2022年。神经点 Catacaustics：反射的新视角合成。*ACM Transactions on Graphics (SIGGRAPH Asia Conference Proceedings)* 41, 6 (2022年)，第201页。http://www-sop.inria.fr/reves/Basilic/2022/KLRJD22 Georgios Kopanas、Julien Philip、Thomas Leimkühler和George Drettakis。2021年。基于点云的神经渲染与逐视图优化。*Computer Graphics Forum* 40, 4 2021 29 43 10 1111 14339 ( )，第1–12页。https://doi.org/ . /cgf. Samuli Laine和Tero Karras。2011年。GPU上的高性能软件光栅化。收录于《ACM SIGGRAPH高性能图形研讨会论文集》。第79–88页。

Christoph Lassner和Michael Zollhofer。2021。Pulsar：基于球体的神经渲染。在*IEEE/CVF*计算机视觉与模式识别会议论文集（*CVPR*）。1440–1449。Marc Levoy和Pat Hanrahan。1996。光场渲染。在 第*23*届计算机图形学和交互技术年会的论文集。31–42。Stephen Lombardi，Tomas Simon，Gabriel Schwartz，Michael Zollhoefer，Yaser Sheikh，和Jason Saragih。2021。用于高效神经渲染的体积基元混合。 *ACM*图形学汇刊 *(TOG)* 40, 4 (2021)，1–13。Duane G Merrill和Andrew S Grimshaw。2010。重新审视GPGPU流架构的排序。在 第*19*届并行架构与编译技术国际会议论文集。545–546。Ben Mildenhall，Pratul P. Srinivasan，Matthew Tancik，Jonathan T. Barron，Ravi Ra- mamoorthi，和Ren Ng。2020。NeRF：将场景表示为神经辐射场以进行视图合成。在 *ECCV*。Thomas Müller，Alex Evans，Christoph Schied，和Alexander Keller。2022。具有多分辨率哈希编码的即时神经图形基元。 *ACM Trans. Graph.* 41, 4, 文章102 (2022年7月)，15页。https://doi.org/10.1145/3528223. 3530127 Eric Penner和Li Zhang。2017。用于视图合成的软3D重建。 *ACM*图形学汇刊 *(TOG)* 36, 6 (2017)，1–11。Hanspeter Pfister，Matthias Zwicker，Jeroen van Baar，和Markus Gross。2000。Surfels：作为渲染基元的表面元素。在 第*27*届计算机图形学和交互技术年会论文集 *(SIGGRAPH '00)*。ACM Press/Addison-Wesley Publishing Co., USA, 335–342。https://doi.org/10.1145/ 344779.344936 Christian Reiser，Songyou Peng，Yiyi Liao，和Andreas Geiger。2021。KiloNeRF：使用数千个微小的MLP加速神经辐射场。在 国际计算机视觉会议 *(ICCV)*。Liu Ren，Hanspeter Pfister，和Matthias Zwicker。2002。对象空间EWA表面喷溅：一种用于高质量点渲染的硬件加速方法。 计算机图形学论坛 21 (2002)。Helge Rhodin，Nadia Robertini，Christian Richardt，Hans-Peter Seidel，和Christian Theobalt。2015。具有可微可见性的多功能场景模型，应用于生成姿态估计。在 *IEEE*国际计算机视觉会议论文集。765–773。Gernot Riegler和Vladlen Koltun。2020。自由视图合成。在 欧洲计算机视觉会议。Springer，623–640。Darius Rückert，Linus Franke，和Marc Stamminger。2022。ADOP：近似可微的单像素点渲染。 *ACM Trans. Graph.* 41, 4, 文章99 (2022年7月)，14页。https://doi.org/10.1145/3528223.3530122 Miguel Sainz和Renato Pajarola。2004。基于点的渲染技术。 计算机与图形学 28, 6 (2004)，869–879。https://doi.org/10.1016/j.cag.2004.08.014 Johannes Lutz Schönberger和Jan-Michael Frahm。2016。运动结构重访。在 计算机视觉与模式识别会议 *(CVPR)*。Markus Schütz，Bernhard Kerbl，和Michael Wimmer。2022。实时软件光栅化2亿个点。 *Proc. ACM Comput. Graph. Interact. Tech.* 5, 3, 文章24 (2022年7月)，17页。https://doi.org/10.1145/3543863 Vincent Sitzmann，Justus Thies，Felix Heide，Matthias Nießner，Gordon Wetzstein，和Michael Zollhofer。2019。Deepvoxels：学习持久的3D特征嵌入。在 *IEEE/CVF*计算机视觉与模式识别会议论文集。2437–2446。Noah Snavely，Steven M Seitz，和Richard Szeliski。2006。Photo tourism：在3D中探索照片集合。在 *Proc. SIGGRAPH*。Carsten Stoll，Nils Hasler，Juergen Gall，Hans-Peter Seidel，和Christian Theobalt。2011。使用高斯分布身体模型进行快速关节运动跟踪。在 *2011*国际计算机视觉会议。IEEE, 951–958。Cheng Sun，Min Sun，和Hwann-Tzong Chen。2022。直接体素网格优化：辐射场重建的超快收敛。在 *CVPR*。Towaki Takikawa，Alex Evans，Jonathan Tremblay，Thomas Müller，Morgan McGuire，Alec Jacobson，和Sanja Fidler。2022。可变比特率神经场。在 *ACM SIG- GRAPH 2022*会议论文集。1–9。Towaki Takikawa，Joey Litalien，Kangxue Yin，Karsten Kreis，Charles Loop，Derek Nowrouzezahrai，Alec Jacobson，Morgan McGuire，和Sanja Fidler。2021。神经几何细节层次：具有隐式3D形状的实时渲染。(2021)。Ayush Tewari，Justus Thies，Ben Mildenhall，Pratul Srinivasan，Edgar Tretschk，W Yifan，Christoph Lassner，Vincent Sitzmann，Ricardo Martin-Brualla，Stephen Lombardi，等。2022。神经渲染的进展。在 计算机图形学论坛，第41卷。Wiley Online Library, 703–735。Justus Thies，Michael Zollhöfer，和Matthias Nießner。2019。延迟神经渲染：使用神经纹理的图像合成。 *ACM*图形学汇刊 *(TOG)* 38, 4 (2019)，1–12。Angtian Wang，Peng Wang，Jian Sun，Adam Kortylewski，和Alan Yuille。2023。VoGE：一种使用高斯椭球体进行合成分析的微分体积渲染器。在 第十一届学习表示国际会议。https://openreview.net/forum?id=AdPJb9cud_Y

Olivia Wiles, Georgia Gkioxari, Richard Szeliski, 和 Justin Johnson. 2020. Synsin: 从单张图像进行端到端视图合成. 在 *IEEE/CVF* 计算机视觉与模式识别会议论文集. 7467–7477. Xiuchao Wu, Jiamin Xu, Zihan Zhu, Hujun Bao, Qixing Huang, James Tompkin, 和 Weiwei Xu. 2022. 可扩展的神经室内场景渲染. *ACM* 图形学汇刊 *(TOG)* (2022). Yiheng Xie, Towaki Takikawa, Shunsuke Saito, Or Litany, Shiqin Yan, Numair Khan, Federico Tombari, James Tompkin, Vincent Sitzmann, 和 Srinath Sridhar. 2022. 视觉计算及其他领域的神经场. 在 计算机图形学论坛, Vol. 41. Wiley Online Library, 641–676. Qiangeng Xu, Zexiang Xu, Julien Philip, Sai Bi, Zhixin Shu, Kalyan Sunkavalli, 和 Ulrich Neumann. 2022. Point-nerf: 基于点的神经辐射场. 在 *IEEE/CVF* 计算机视觉与模式识别会议论文集. 5438–5448. Wang Yifan, Felice Serena, Shihao Wu, Cengiz Öztireli, 和 Olga Sorkine-Hornung. 2019. 可微分的表面喷涂用于基于点的几何处理. *ACM* 图形学汇刊 *(TOG)* 38, 6 (2019), 1–14. Alex Yu, Ruilong Li, Matthew Tancik, Hao Li, Ren Ng, 和 Angjoo Kanazawa. 2021. PlenOctrees 用于神经辐射场的实时渲染. 在 *ICCV*. Qiang Zhang, Seung-Hwan Baek, Szymon Rusinkiewicz, 和 Felix Heide. 2022. 可微分的基于点的辐射场用于高效视图合成. 在 ***SIGGRAPH Asia 2022*** 会议论文集(Daegu, 大韩民国) *(SA '22)*. Association for Computing Machinery, New York, NY, USA, Article 7, 12 页. https://doi.org/10. 1145/3550469.3555413 Tinghui Zhou, Shubham Tulsiani, Weilun Sun, Jitendra Malik, 和 Alexei A Efros. 2016. 基于外观流的视图合成. 在 欧洲计算机视觉会议. Springer, 286–301. Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, 和 Markus Gross. 2001a. EWA 体积喷涂. 在 ***2001*** 年可视化会议论文集, ***VIS'01***. IEEE, 29–538. Matthias Zwicker, Hanspeter Pfister, Jeroen van Baar, 和 Markus Gross. 2001b. 表面喷涂. 在 第 ***28*** 届计算机图形学与交互技术会议论文集 ***(SIGGRAPH '01)***. Association for Computing Machinery, New York, NY, USA, 371–378. https://doi.org/10.1145/383259.383300

梯度计算细节

回想 Σ/Σ' 是高斯的世界/视图空间协方差矩阵， q 是旋转， s 是缩放， W 是视图变换， J 是投影变换的仿射近似的雅可比矩阵。我们可以应用链式法则来找到关于缩放和旋转的导数：

$$\frac{d\Sigma'}{ds} = \frac{d\Sigma'}{d\Sigma}\frac{d\Sigma}{ds} \tag{8}$$

and

$$\frac{d\Sigma'}{dq} = \frac{d\Sigma'}{d\Sigma}\frac{d\Sigma}{dq} \tag{9}$$

使用 $U = JW$ 和 Σ 简化公式5，其中 $'$ 表示 $U\Sigma U^T$ 的（对称）左上角 2×2 矩阵，用下标表示矩阵元素，我们可以找到偏导数 $\frac{\partial \Sigma'}{\partial \Sigma_{ij}} = \begin{pmatrix} U_{1,i}U_{1,j} & U_{1,i}U_{2,j} \\ U_{1,j}U_{2,i} & U_{2,i}U_{2,j} \end{pmatrix}$。

接下来，我们求解 $\frac{d\Sigma}{ds}$ 和 $\frac{d\Sigma}{dq}$ 的导数。由于 $\Sigma = RSS^TR^T$，我们可以计算 $M = RS$ 并重写 $\Sigma = MM^T$。因此，我们可以写出 $\frac{d\Sigma}{ds} = \frac{d\Sigma}{dM}\frac{dM}{ds}$ 和 $\frac{d\Sigma}{dq} = \frac{d\Sigma}{dM}\frac{dM}{dq}$。由于协方差矩阵 Σ （及其梯度）是对称的，共享的第一部分可以通过 $\frac{d\Sigma}{dM} = 2M^T$ 紧凑地找到。对于缩放，我们进一步有 $\frac{\partial M_{i,j}}{\partial s_k} = \begin{cases} R_{i,k} \text{ 如果 j = k} \\ 0 \text{ 否则} \end{cases}$。为了推导旋转的梯度，我们回忆从单位四元数 q （实部为 q_r，虚部为 q_i, q_j, q_k）转换为旋转矩阵 R 的转换：

$$R(q) = 2\begin{pmatrix} \frac{1}{2} - (q_j^2 + q_k^2) & (q_iq_j - q_rq_k) & (q_iq_k + q_rq_j) \\ (q_iq_j + q_rq_k) & \frac{1}{2} - (q_i^2 + q_k^2) & (q_jq_k - q_rq_i) \\ (q_iq_k - q_rq_j) & (q_jq_k + q_rq_i) & \frac{1}{2} - (q_i^2 + q_j^2) \end{pmatrix} \tag{10}$$

因此，我们找到以下关于 q 分量的梯度：

$$\frac{\partial M}{\partial q_r} = 2\begin{pmatrix} 0 & -s_yq_k & s_zq_j \\ s_xq_k & 0 & -s_zq_i \\ -s_xq_j & s_yq_i & 0 \end{pmatrix}, \quad \frac{\partial M}{\partial q_i} = 2\begin{pmatrix} 0 & s_yq_j & s_zq_k \\ s_xq_j & -2s_yq_i & -s_zq_r \\ s_xq_k & s_yq_r & -2s_zq_i \end{pmatrix}$$
$$\frac{\partial M}{\partial q_j} = 2\begin{pmatrix} -2s_xq_j & s_yq_i & s_zq_r \\ s_xq_i & 0 & s_zq_k \\ -s_xq_r & s_yq_k & -2s_zq_j \end{pmatrix}, \quad \frac{\partial M}{\partial q_k} = 2\begin{pmatrix} -2s_xq_k & -s_yq_r & s_zq_i \\ s_xq_r & -2s_yq_k & s_zq_j \\ s_xq_i & s_yq_j & 0 \end{pmatrix} \tag{11}$$

计算四元数归一化的梯度很简单 .

B 优化与稠密化算法

我们的优化与稠密化算法总结在算法1中。

算法1 优化与稠密化

w, h: 训练图像的宽度和高度

```
M ←SfM点                                              ▷ 位置
S, C, A ← 初始化属性()                                 ▷ 协方差，颜色，不透明度
i ← 0                                                 ▷ 迭代次数
while未收敛 do
    V, Î ←采样训练视图()                               ▷ 相机 V 和 图像
    I ← 光栅化(M, S, C, A, V)                         ▷ 算法2
    L ← Loss(I, Î)                                    ▷ 损失
    M, S, C, A ← Adam优化器(∇L)                       ▷ 反向传播与步进
    如果 IsRefinementIteration(i)则
        对于所有 高斯分布 (μ, Σ, c, α)在 (M, S, C, A)中执行
            如果 α < ϵ 或 IsTooLarge(μ, Σ) 则          ▷ 剪枝
                RemoveGaussian()
            结束如果
            如果 ∇_p L > τ_p 则                        ▷ Densification
                如果 ||S|| > τ_S 则                    ▷ 过重建
                    分割Gaussian(μ, Σ, c, α)
                else                                  ▷ 欠重建
                    克隆高斯(μ, Σ, c, α)
                end if
            end if
        end for
    end if
    i ← i + 1
end while
```

C 光栅化详情

排序。 我们的设计基于小高斯分布高负载的假设，并通过在每帧开始时使用基数排序对高斯分布进行一次排序来优化这一点。我们将屏幕分割成16x16像素的瓦片（或桶）。我们通过实例化每个高斯分布在每个它重叠的 16×16 瓦片中来创建每个瓦片的高斯分布列表。这导致需要处理的高斯分布数量适度增加，但这一点通过更简单的控制流和优化的GPU基数排序 [Merrill和 Grimshaw 2010]的高并行性得到摊销。我们为每个高斯分布实例分配一个最多64位的密钥，其中低32位编码其投影深度，高位编码重叠瓦片的索引。索引的确切大小取决于当前分辨率可以容纳多少个瓦片。深度排序因此可以直接并行解决所有高斯分布，只需一次基数排序。之后

排序后，我们可以通过识别排序数组中具有相同瓦片ID的范围的起始和结束来有效地生成每个瓦片的要处理的高斯分布列表。这是并行完成的，为每个64位数组元素启动一个线程来将其高位32位与其两个邻居进行比较。与 [Lassner和Zollhofer 2021]，相比，我们的光栅化完全消除了顺序图元处理步骤，并生成了在正向传递期间遍历的更紧凑的每个瓦片列表。我们在算法2中展示了光栅化方法的高级概述。

算法 2三维高斯的光栅化

w, h: 要光栅化的图像的宽度和高度
M, S: 世界空间中的高斯均值和协方差
C, A: 高斯颜色和不透明度
V: 当前相机的视图配置

function 光栅化(w, h, M, S, C, A, V)
  CullGaussian(p, V) ▷ Frustum Culling
  M' S' ← ScreenspaceGaussians(M,S, V) ▷ 变换
  T ← CreateTiles(w, h)
  L, K ← DuplicateWithKeys(M', T) ▷ 索引和键
  SortByKeys(K, L) ▷ 全局排序
  R ← 识别瓦片范围(T, K)
  I ← **0** ▷ 初始化画布
  对所有瓦片 t **在** I **中做**
    对所有像素 i **在** t **中做**
      r ← 获取瓦片范围(R, t)
      $I[i]$ ← 按顺序混合(i, L, r, K, M', S', C, A)
    结束循环
  结束循环
  返回 I
结束函数

数值稳定性。 在反向传播过程中，我们通过反复将正向传播累积的不透明度除以每个高斯函数的 α来重建梯度计算所需的中间不透明度值。如果原始实现，这个过程容易导致数值不稳定（例如，除以0）。为了解决这个问题，在正向和反向传播中，我们都会跳过与 $\alpha < \epsilon$ 的任何混合更新（我们选择 ϵ 作为 $\frac{1}{255}$），并且将 α 限制在 0。99 以上。最后，**在** 高斯被包含在正向光栅化传递之前，我们会计算如果我们包含它的话累积的不透明度，并且停止前后混合**在** 它超过 0。9999 之前。

D 场景级误差指标

表4至表9列出了我们针对所有考虑的技术和真实场景收集的各种误差指标。我们列出了复制的Mip-NeRF360数据以及用于生成论文中图像的运行结果；这些在完整Mip-NeRF360数据集上的平均值分别为PSNR 27.58、SSIM 0.790和LPIPS 0.240。

表4。Mip-NeRF360场景的SSIM分数。† 复制自原始论文。

	自行车	花朵	花园	树桩	树丘	room	计数器	厨房	盆景
Plenoxels	0.496	0.431	0.6063	0.523	0.509	0.8417	0.759	0.648	0.814
INGP-Base	0.491	0.450	0.649	0.574	0.518	0.855	0.798	0.818	0.890
INGP-Big	0.512	0.486	0.701	0.594	0.542	0.871	0.817	0.858	0.906
Mip-NeRF360†	0.685	0.583	0.813	0.744	0.632	0.913	0.894	0.920	**0.941**
Mip-NeRF360	0.685	0.584	0.809	0.745	0.631	0.910	0.892	0.917	0.938
Ours-7k	0.675	0.525	0.836	0.728	0.598	0.884	0.873	0.900	0.910
Ours-30k	**0.771**	**0.605**	**0.868**	**0.775**	**0.638**	**0.914**	**0.905**	**0.922**	0.938

表 5. Mip-NeRF360 场景的 PSNR 分数。† 从原始论文复制。

	自行车	花朵	花园	树桩	树丘	room	计数器	厨房	盆景
Plenoxels	21.912	20.097	23.4947	20.661	22.248	27.594	23.624	23.420	24.669
INGP-Base	22.193	20.348	24.599	23.626	22.364	29.269	26.439	28.548	30.337
INGP-Big	22.171	20.652	25.069	23.466	22.373	29.690	26.691	29.479	30.685
Mip-NeRF360†	24.37	**21.73**	26.98	26.40	22.87	**31.63**	**29.55**	**32.23**	**33.46**
Mip-NeRF360	24.305	21.649	26.875	26.175	**22.929**	31.467	29.447	31.989	33.397
Ours-7k	23.604	20.515	26.245	25.709	22.085	28.139	26.705	28.546	28.850
Ours-30k	**25.246**	21.520	**27.410**	**26.550**	22.490	30.632	28.700	30.317	31.980

表 6。Mip-NeRF360 场景的 LPIPS 分数。† 从原始论文复制。

	自行车	花草树丛				room	厨房盆景		
Plenoxels	0.506	0.521	0.3864	0.503	0.540	0.4186	0.441	0.447	0.398
INGP-Base	0.487	0.481	0.312	0.450	0.489	0.301	0.342	0.254	0.227
INGP-Big	0.446	0.441	0.257	0.421	0.450	0.261	0.306	0.195	0.205
Mip-NeRF360†	0.301	0.344	0.170	0.261	0.339	**0.211**	**0.204**	**0.127**	**0.176**
Mip-NeRF360	0.305	0.346	0.171	0.265	0.347	0.213	0.207	0.128	0.179
Ours-7k	0.318	0.417	0.153	0.287	0.404	0.272	0.254	0.161	0.244
Ours-30k	**0.205**	**0.336**	**0.103**	**0.210**	**0.317**	0.220	**0.204**	0.129	0.205

表7. 坦克与寺庙和Deep Blending场景的SSIM分数。

	卡车	火车	约翰逊博士	游戏室
Plenoxels	0.774	0.663	0.787	0.802
INGP-Base	0.779	0.666	0.839	0.754
INGP-Big	0.800	0.689	0.854	0.779
Mip-NeRF360	0.857	0.660	**0.901**	0.900
Ours-7k	0.840	0.694	0.853	0.896
Ours-30k	**0.879**	**0.802**	0.899	**0.906**

表8. 坦克与寺庙和深度融合场景的PSNR得分。

	卡车	火车	约翰逊博士	游戏室
Plenoxels	23.221	18.927	23.142	22.980
INGP-Base	23.260	20.170	27.750	19.483
INGP-Big	23.383	20.456	28.257	21.665
Mip-NeRF360	24.912	19.523	**29.140**	29.657
Ours-7k	23.506	18.892	26.306	29.245
Ours-30K	**25.187**	**21.097**	28.766	**30.044**

表9. 坦克与寺庙和深度融合场景的LPIPS分数。

	卡车	火车	约翰逊博士	游戏室
Plenoxels	0.335	0.422	0.521	0.499
INGP-Base	0.274	0.386	0.381	0.465
INGP-Big	0.249	0.360	0.352	0.428
Mip-NeRF360	0.159	0.354	**0.237**	0.252
Ours-7k	0.209	0.350	0.343	0.291
Ours-30k	**0.148**	**0.218**	0.244	**0.241**